Q-Neuro

What I found when I spent a year trying to invent a better neural network, and failed nineteen times

Samyar Shafiee · independent, Toronto · built and measured on one 8 GB MacBook Air

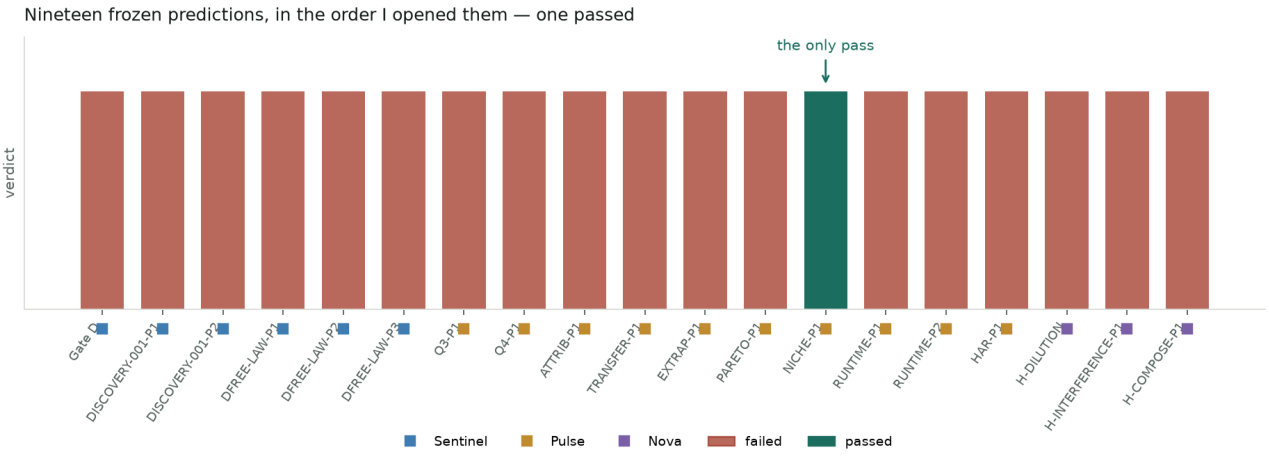
Nineteen frozen predictions · one passed · 39 preserved failures

What I found when I spent a year trying to invent a better neural network, and failed nineteen times

I am a high school student. I do this as a hobby, on an 8 GB MacBook Air, and I want to be upfront about that because it shapes everything here: every experiment had to fit on a laptop, and I had no supervisor to stop me from fooling myself. So I built tools to stop myself instead.

This is the report of what happened. It is mostly a record of things that did not work, which is a strange thing to write up — but the not-working is the part I am confident about, and I would rather hand you something small and true than something big and shaky.

The one-sentence summary: I froze nineteen predictions before looking at their results, one of them passed, and the thing that passed is smaller than I hoped.



Every prediction I froze, in the order I opened them

---

1. What I was actually trying to do

Here is the question that got me started.

A neural network is a machine that turns numbers into other numbers. You give it examples, it adjusts millions of internal knobs until its answers match, and then — hopefully — it works on examples it has never seen. That last part is the mystery. Why should it?

I wanted to find a **better way for these machines to compute**. Not a better tuning trick. Not another 0.4% on a benchmark. An actually different way of arranging the computation, so that the same amount of hardware buys more capability.

That is an absurdly ambitious goal for a laptop. I knew that. But I figured the interesting part was not whether I would succeed — it was whether I could build a process honest enough that I would *know* if I failed. Most of what follows is that process working.

The project went through four eras. I named them afterwards, because at the time I mostly did not know what era I was in.

Era	What I believed	How it ended
Helix	Complex numbers give networks a real advantage	Overturned by a control I built myself

Era	What I believed	How it ended
Sentinel	I need tools that catch false discoveries	The tools worked. That is the durable part.
Pulse	Models should choose how long to think	One result survived, with a hard ceiling
Nova	Start clean, search for a new principle	Found nothing new. Best idea was from 2014.

---

2. Helix: the idea I was in love with

The intuition

Normal networks store a number at each unit. What if they stored a number **and a phase** — a complex number?

Phase is what makes waves interesting. Two waves can reinforce each other or cancel out. That is interference, and it is how light and sound carry structured information through noise. If a task needs the network to combine evidence in an order-sensitive way, phase seems like exactly the right tool. It felt beautiful.

And it worked! On my simulator, the complex model beat every control across five unseen worlds and four difficulty levels, by **+0.054 to +0.063**. I isolated the mechanisms: ordered composition contributed **+0.232**, phase-sensitive readout **+0.104**.

Any one of those numbers would have made me very happy. That is exactly the problem.

The controls that killed it, one at a time

What I claimed	What killed it
Complex models need less data	A properly tuned GRU hit 0.920 where complex hit 0.699
My calibration transfers under shift	It transferred for nobody — it made every model worse
Complex represents ambiguity better	Complex scored 2.581 where plain real scored 1.148 (lower is better)
Complex states uniquely encode structure	GRU internal states were more informative on every factor

Notice the pattern: **each claim died against a better baseline, not a better idea.** I had been comparing my model against opponents I had not bothered to tune.

The one that finished it

Here is a fact about complex numbers I should have taken seriously earlier. Any complex matrix `M = A + iB` can be written as a real matrix:

```
R(M) = [ A  -B ]
       [ B   A ]
```

Which means **every complex network has an exact real twin that computes the identical function.** Not approximately. Identically.

So I built the twin and compared them.

> *Top-1 predictions matched in **all 1,920** held-out test cases. Across 2,880 more, complex won **zero** of them. The confidence interval for complex-minus-best-real was [−0.01325, −0.00457] — entirely below zero.*

The idea the whole project is named after does not work. I spent a long time not wanting that to be true.

The lesson I paid for

> ***A comparison is only as good as your ability to say exactly what is being compared.***

"The real version of my model" is not a specification. It hides four separate choices. Realising that is what created the next era.

---

3. Sentinel: building tools to catch myself

Sentinel is not a model. It is the machinery I built to detect when I am fooling myself. It ended up being the most valuable thing in the project.

Equivalence is a property of a *map*, not of two models

If someone says "these two networks are equivalent," that sentence means nothing until four questions are answered:

- **Which map** turns one into the other?
- **At what level** — identical symbolically? bit-for-bit? only on average?
- **On what domain** — everywhere, or with exceptions?
- **What carries over** — just the weights, or the gradients and optimiser state too?

I built a type system that refuses to certify claims that skip these. If you declare "exactly identical" *and* "except in this region," it raises an error at construction. Certificates can be downgraded, never upgraded.

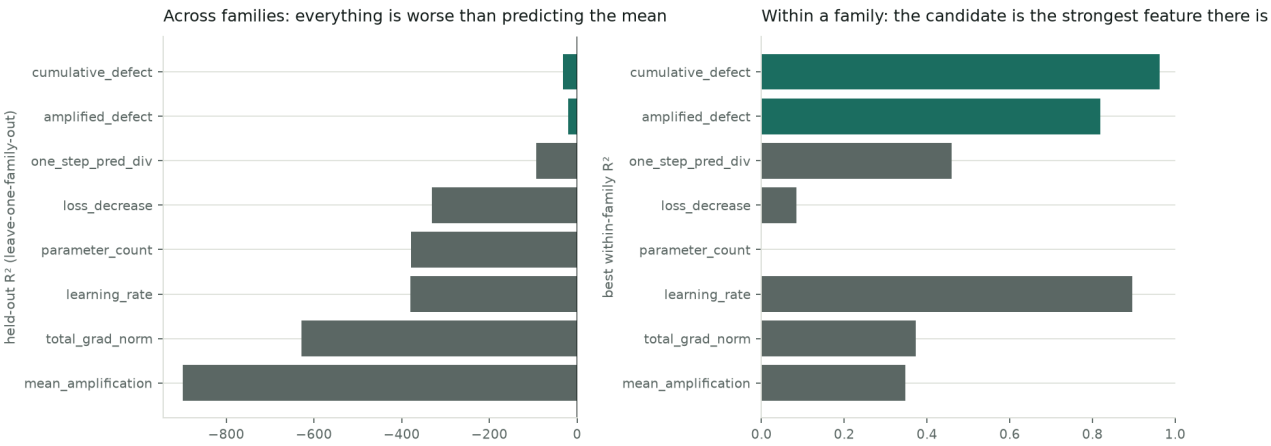
And it immediately caught my own project

Remember the exact-real twin from Helix? It shared coordinates with the complex model. It was literally the same model with two labels on it. **1,478 of my 1,920 "wins" were the model tying with itself.**

The negative result survived — genuinely different real architectures still beat complex, on the other 442 cases — but I had to rewrite the headline.

The big conjecture, and its death

I thought I had a real theory. If you can measure how badly a transformation *fails to commute* with the optimiser, you should be able to predict how far the two models drift apart. I called it the transport-covariance conjecture, froze it, and tested it.

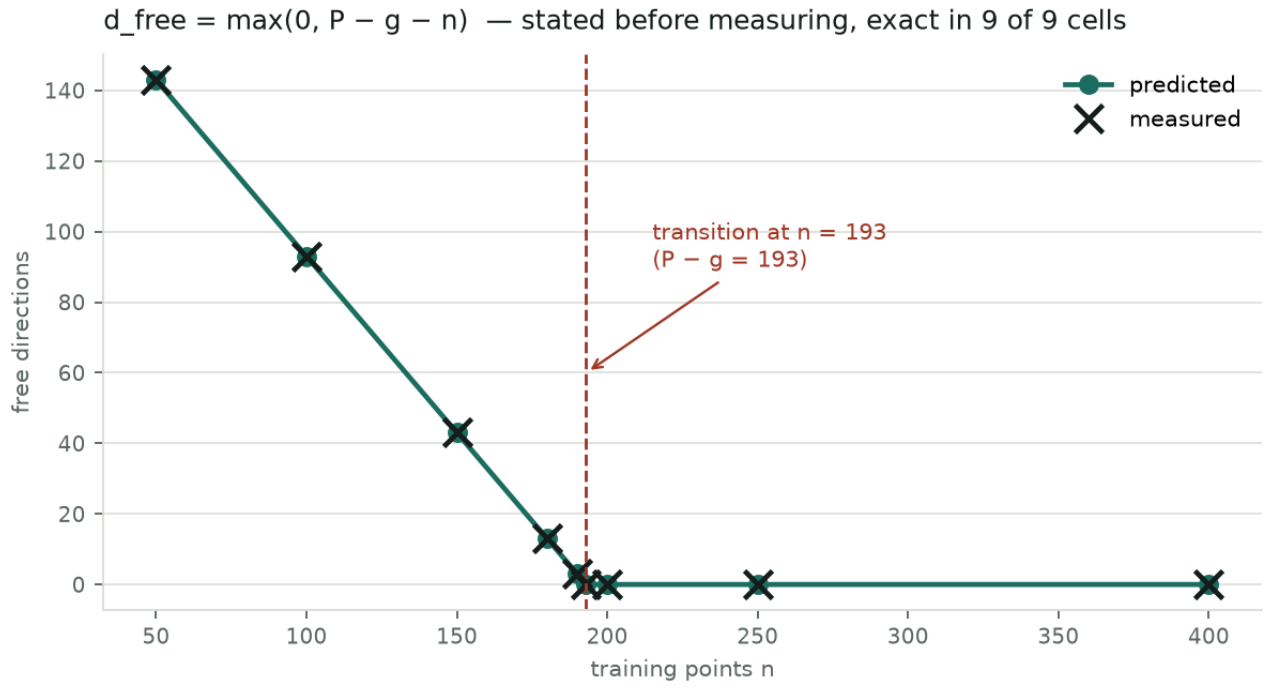


Gate D: the conjecture works within a family and collapses across families

It failed. Held-out R² of **−31.7** — worse than just guessing the average.

And the *way* it failed is interesting. Look at the right panel: within a single family, my candidate is the best feature available (R² 0.962). Across families it collapses, because the families' typical values span **6.5 orders of magnitude**. One family sits at 1e-7 because its map is perfectly conjugate — nothing left to predict — and another sits at 1e-0.6. Fitting one line through both is hopeless.

The one piece of maths that did work



The dimension law: predicted before measuring, exact in 9 of 9

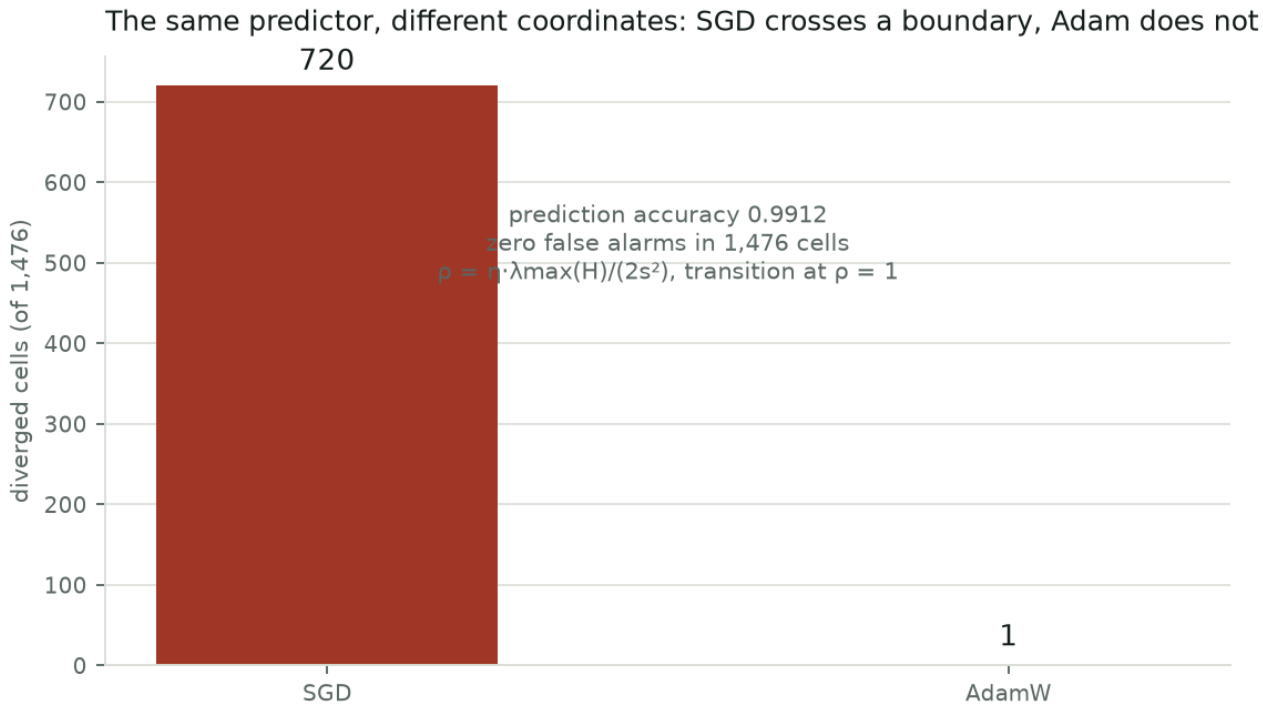
```
d_free = max(0, P − g − n(C−1))
```

`P` is the parameter count, `n` the number of training points, and `g` the number of directions you can move without changing anything — the network's built-in symmetries. Softmax has some; ReLU has more, because scaling one layer up and the next down changes nothing.

I predicted the numbers before measuring them and got **9 out of 9 exactly**, including the moment it hits zero at n = 193.

Why it matters: I had spent seven experiments hunting for "free directions" that improve a model without hurting training. All seven failed. This formula says they were doomed — at my settings, `d_free = 0` exactly. There was nothing to find. That is a much better answer than "it didn't work."

And a beautiful result that also died



The same predictor in different coordinates lands on opposite sides of a cliff

Two models that are *provably the same predictor*, trained identically, and one converges while the other explodes — purely because of the coordinates you wrote it in. Prediction accuracy 0.9912. **Zero** false alarms in 1,476 cases. And a differential prediction I made in advance held: Adam's update is scale-free, so it should not show the cliff, and it does not — 1 divergent case out of 1,476 against SGD's 720.

It is exact for simple objectives. On a real nonlinear network it failed **96 out of 96**. The reason was written into my own frozen prediction beforehand: the formula uses curvature at the starting point, and a real network *moves somewhere flatter* while training.

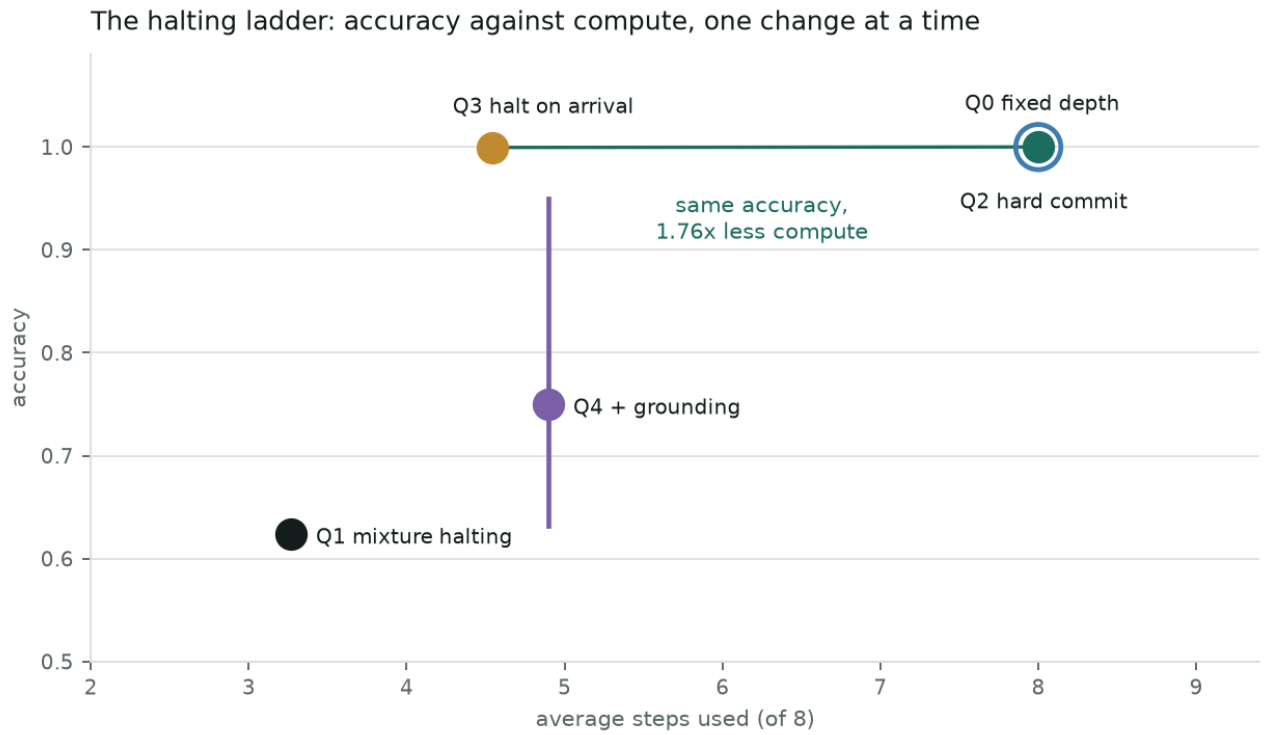
---

4. Pulse: teaching a model to decide how long to think

The idea

Most networks do the same amount of work on every input. That is obviously wasteful — some questions are easy. So: let the model decide when to stop.

I built a task where this genuinely matters. A permutation makes a hidden loop through 24 nodes; the model starts somewhere and has to report how far away the goal is. The only way to know is to actually follow the chain. Easy examples need two steps, hard ones need eight.



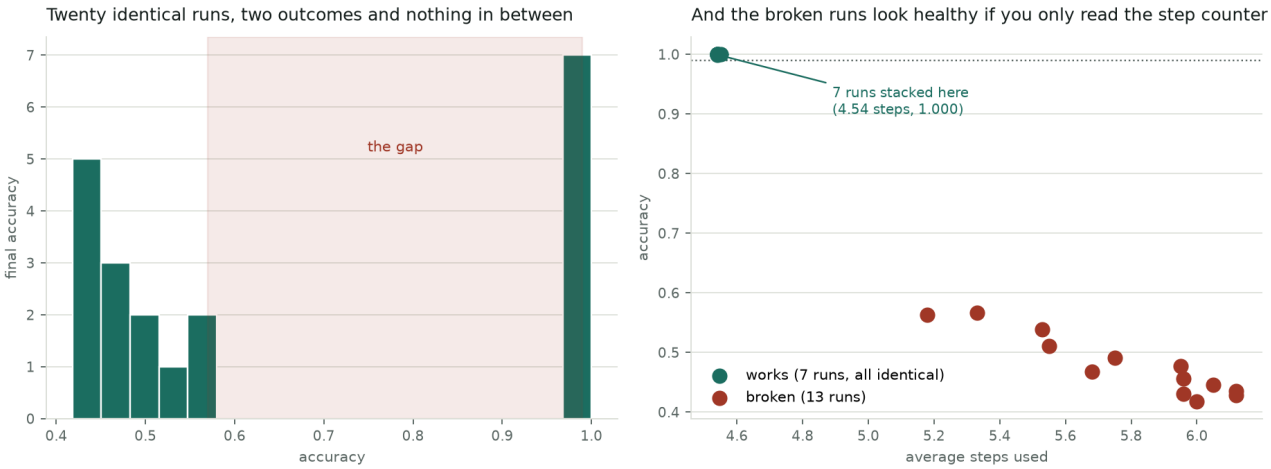
Four ways of deciding when to stop

- **Q0** always uses 8 steps: perfect accuracy, maximum cost.
- **Q1** (the standard approach) collapses to 0.6241 — it stops thinking before it learns to think.
- **Q2** fixes the accuracy but only by using all 8 steps, so it saves nothing.
- **Q3** halts when it *detects arrival*, and lets the step count be the answer: **1.77× less

compute at the same accuracy, with fewer parameters.**

I was thrilled. Then I checked it properly.

The most important thing I learned all year



The same code, twenty times

I ran the identical configuration twenty times with different random seeds. It works about half the time. The other half it lands at 0.42–0.51 accuracy. **There is nothing in between.**

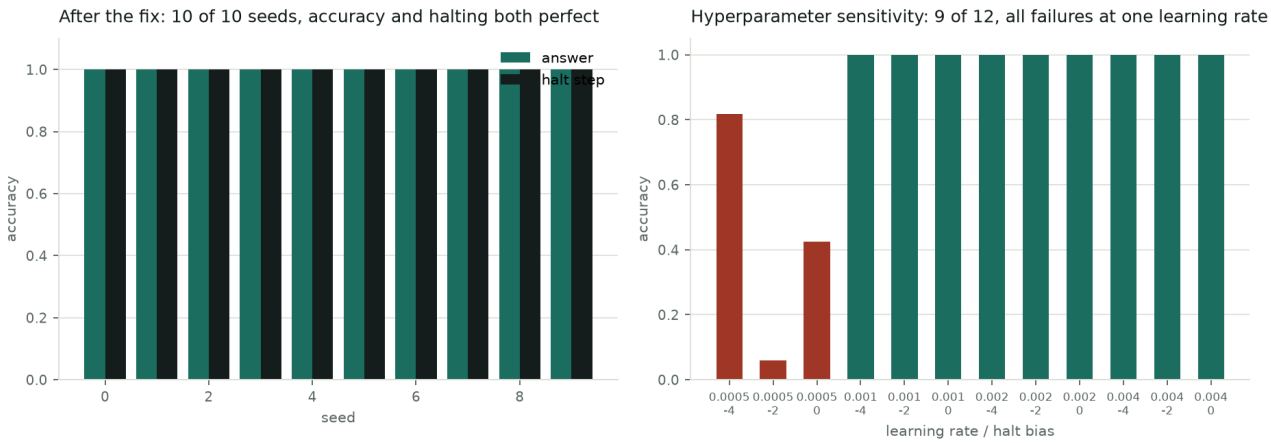
Now look at the right panel, because this is the part that scared me. The broken runs report **5.2–6.1 average steps**. That is a completely reasonable-looking number — under the maximum of 8, varying by example, exactly what a working adaptive model produces.

> ***If you only look at the compute saving, all thirteen broken runs look like successes.***

I had been about to report "1.77× less compute" as a result. It was true for the seeds I happened to run. A compute number without a matching accuracy number and a reliability rate cannot tell you whether the model works.

Fixing it

I diagnosed it by looking at accuracy *split by difficulty*: the failing runs are perfect on short chains and collapse from length 3. The internal state was losing track. Normalising it after each step took reliability from **11 out of 24 to 20 out of 20**.



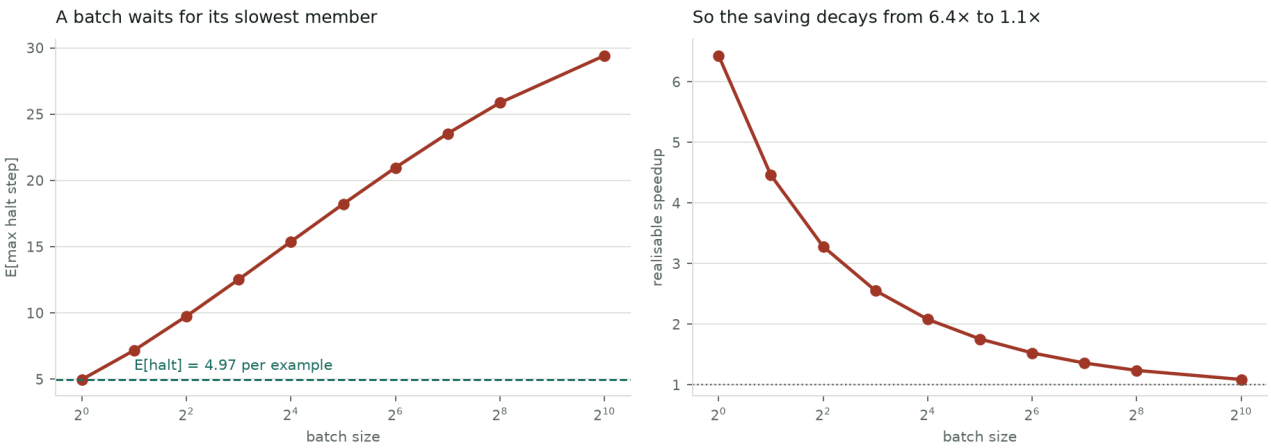
After the fix: every seed, and how sensitive it is to settings

Ten out of ten seeds, perfect on both the answer and the halt step, well calibrated (ECE 0.002), and 9 of 12 hyperparameter settings work — all three failures at one learning rate that was simply too small to converge.

Normalisation is textbook and I claim no credit for it. But there is a genuinely odd thing here: the *same* change **destroys** the fixed-depth model on the *same* task, dropping it from 1.0000 to 0.13–0.25. One change, opposite effects, depending on what the model is reading out.

The ceiling nobody warns you about

Then I tried to bank the speedup and hit something I found genuinely surprising.



A batch waits for its slowest member

If you run examples one at a time, you save what you expect. If you run them in a **batch**, the batch cannot finish until its *slowest* member finishes. So your cost is not the average halt step — it is the **maximum over the batch**.

That is a formula from statistics:

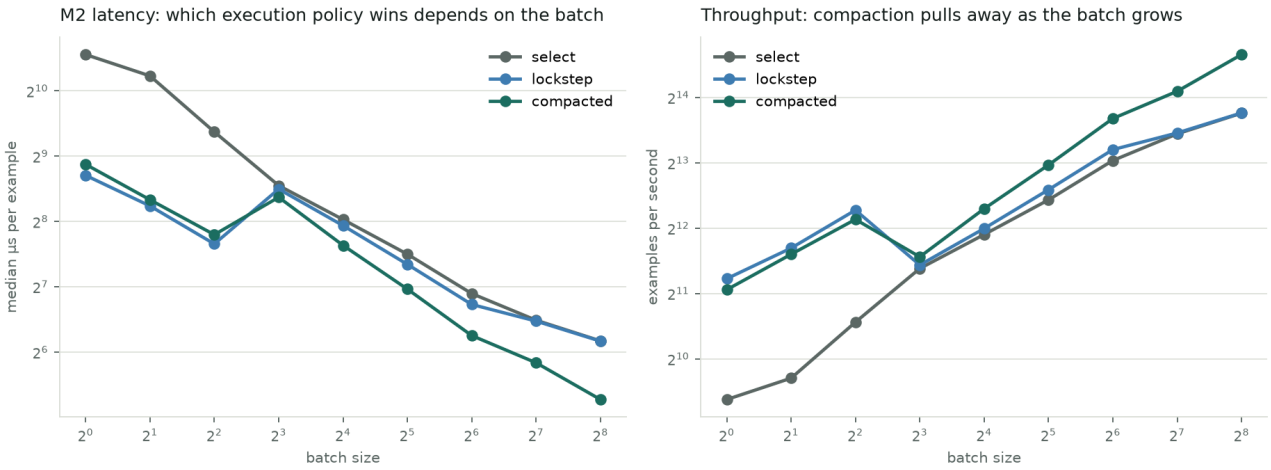
```
E[max] = Σ  k · ( F(k)^n − F(k−1)^n )
```

For my difficulty distribution the average halt is 4.97 steps out of 32. At batch 1 you save 6.4×. At batch 1024 you save **1.09×**. The advantage evaporates.

This applies to every adaptive-compute method I know of, not just mine. I think it is under-discussed and it is why these methods keep disappointing at deployment scale.

I also learned that a saving on paper is not a saving. My first measurement of this showed **1.0×** — no improvement at all — because my batched code ran every step and *then* picked one. The step counter said 4.91. The clock said nothing had changed.

Where it actually wins



Latency and throughput across the whole batch range on my laptop

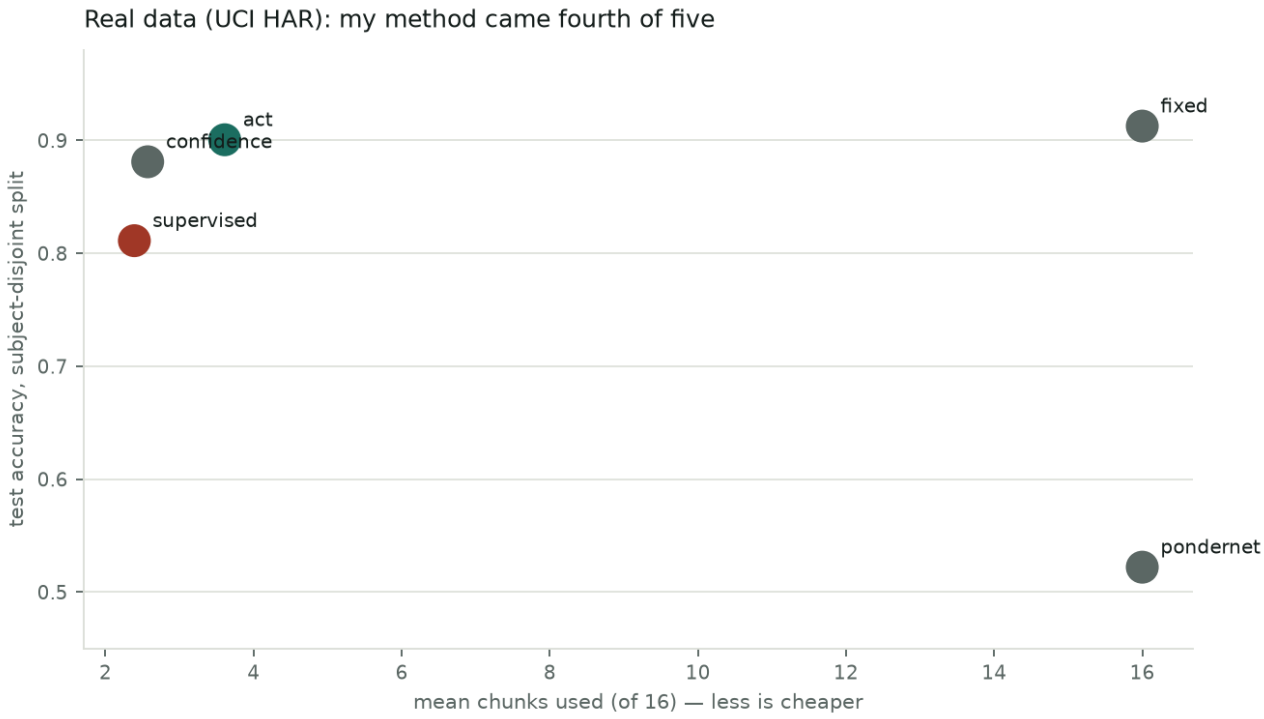
Once I implemented real early exit, and then a technique called active-set compaction (drop finished examples and continue with the rest — this is standard, I did not invent it):

- **batch 1: 3.6× faster** than the matched-accuracy baseline
- **batch 256: 1.86× faster**, throughput from 13,872/s to 25,841/s
- memory from 1,536 KiB down to **457 KiB**

I froze a prediction about this and opened it once on a task family I had not used to develop it. **All four clauses passed** — including the clause that predicted *where the advantage stops working*. That is the only prediction in this entire project that passed as written.

And then real data

Everything above is on tasks I designed. A task you design can accidentally be a task your method is good at. So I got a real dataset — human activity recognition from phone sensors — and used the dataset's *own* train/test split, which separates by person, so I could not tune it.



On real data, my method came fourth of five

Fourth of five. **ACT — a method from 2016 — beat me**, and so did a plain confidence threshold. And mine cost 2.3× more to train, because real data has no built-in "correct time to stop", so I had to train a teacher model first to invent one.

> ***My method only earns its place when the task itself tells you when to stop.*** *Where it does, it is excellent and reliable. Where it does not, simpler things win.*

That is a real result and it is a narrowing, not a victory.

---

5. Nova: starting over, properly

For the last era I threw out the architecture entirely and kept only the tools. New question: can I find a genuinely new way to compute?

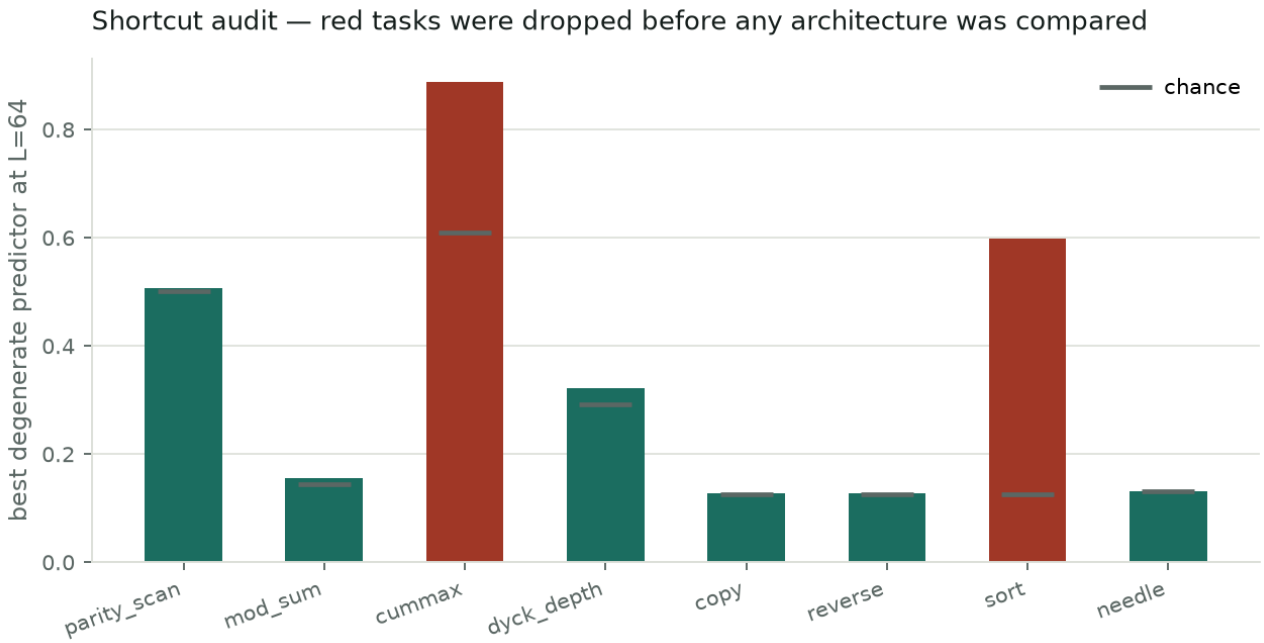
Building the test before the theory

My previous mistake was spending weeks perfecting theories about bad architectures. So this time I built the measuring instrument first: eight tasks where I know the correct algorithm — parity, modular sums, copying, reversing, sorting, associative lookup — trained on short inputs and tested on inputs **four times longer**.

Length extrapolation is a good test because a model that genuinely learned the *procedure* keeps working, and a model that memorised the training lengths does not.

The audit that saved me

Before comparing anything, I asked: what score can you get **without doing the task at all**?

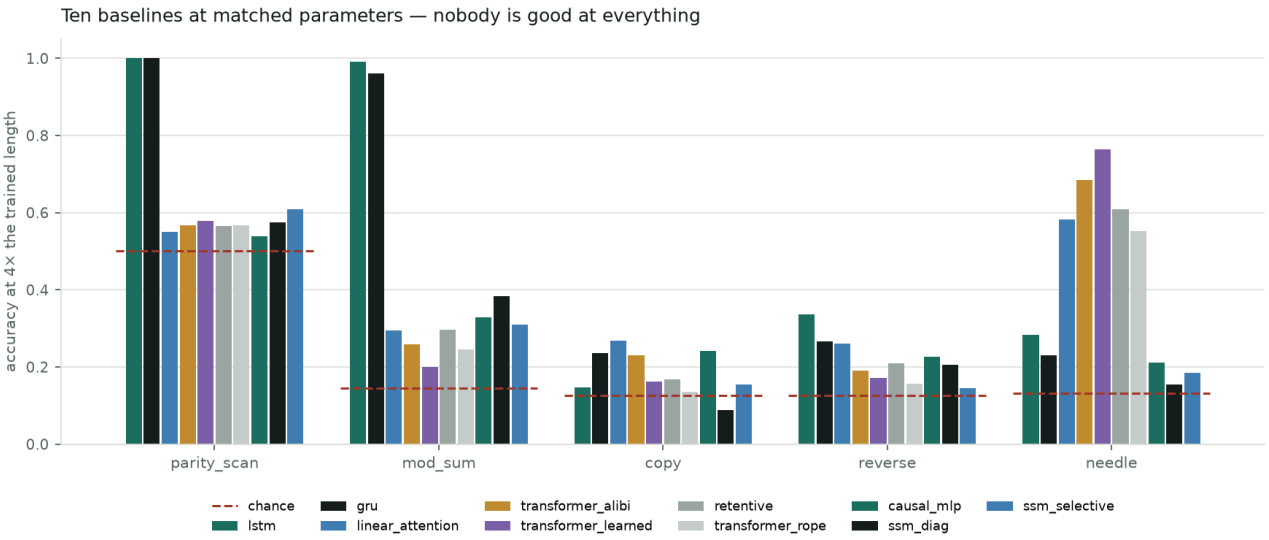


Two of my own tasks turned out to be broken

Just guessing based on position gets **0.887 on cummax** and **0.598 on sort**. Those tasks are useless — you can score well without computing anything. I dropped both.

This mattered immediately: my deliberately-weak control model had scored **0.917 on sort**. Without the audit I would have reported that as a discovery.

The baselines refused to be easy



Ten established architectures, matched for size

A clean pattern emerged that I did not expect to be so stark:

- **Recurrent networks track state and extrapolate perfectly** — LSTM gets **1.000** on parity and

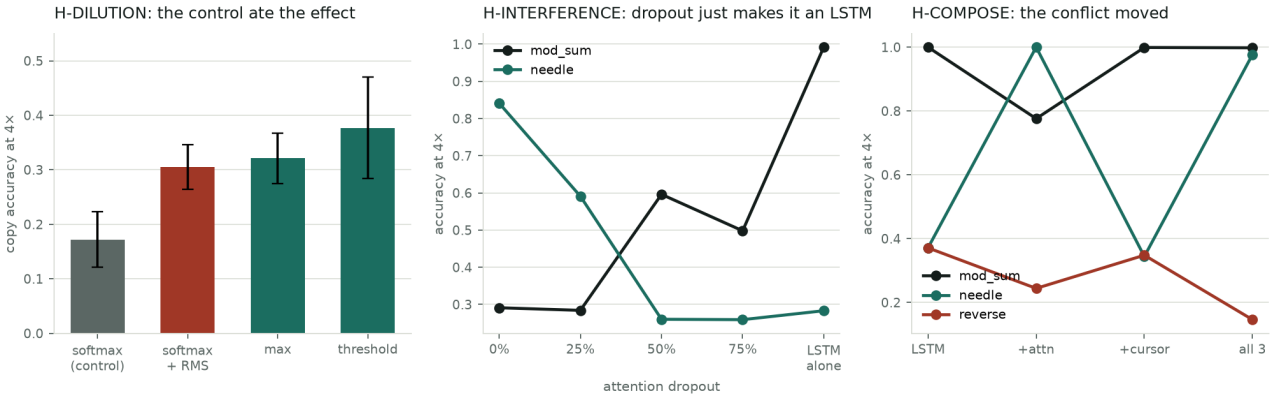
0.992 on modular sum at four times the trained length.

- **Attention retrieves and extrapolates on that** — transformers get up to 0.764 on lookup.
- **Neither does both.** LSTM gets 0.283 on lookup; transformers get 0.20–0.39 on modular sum.

- **Nothing at all extrapolates on copy or reverse.**

The obvious fix is the linear-attention family — recurrent memory that is also searchable. I implemented it **on purpose as a baseline** so I could not later "discover" it. It came out mediocre at both ends. So the gap was real.

Three hypotheses, three deaths



All three, and how each one died

Hypothesis 1 — dilution. Attention divides by a sum over *all* positions, so adding irrelevant items changes the answer. I built a version that ignores non-matching items. The property is real and I measured it: read drift of 0.236 versus softmax's 0.724, three times better.

It made no difference. When I gave plain softmax the *same* extra normalisation my version needed, that alone captured the whole apparent gain (copy 0.172 → 0.305, versus my 0.321).

I also found two bugs on the way. The first version of my "new" operator divided by the sum at the end — which is **algebraically exactly softmax**. It produced numbers identical to the control to three decimal places. That is how I caught it, and until then I had not tested my hypothesis at all.

> ***An operator can genuinely have a property without the property mattering.*** *Those are two different questions and only a control tells them apart.*

Hypothesis 2 — interference. An LSTM alone gets 0.992 on modular sum. Add attention alongside it and it collapses. Was attention drowning out a working recurrence, or preventing it from ever learning?

I answered it by switching branches off at test time, no retraining. Turning attention off makes it **worse** (0.291 → 0.157). Turning the recurrence off is equally bad. Neither works alone — the model found a joint solution that needs both and does not extrapolate.

My fix was to handicap the attention branch during training. All four clauses failed. Dropout does not resolve the conflict; it just slides the model along a trade-off until it *is* an LSTM again.

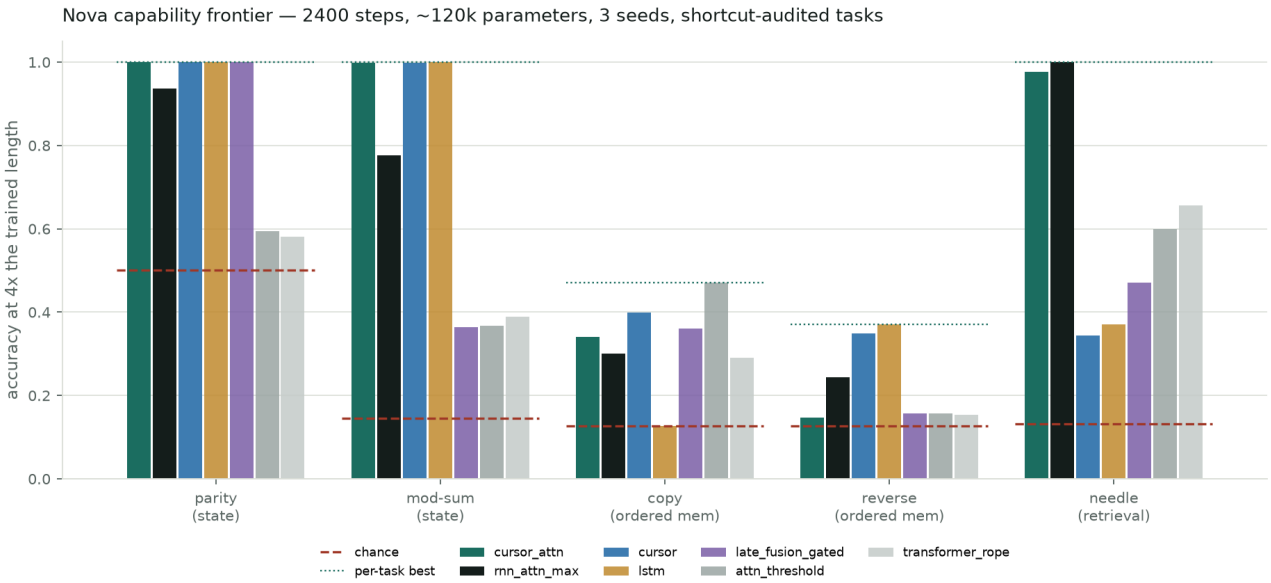
And here I caught myself again. The dramatic version of this effect was partly an artifact of training for too few steps. At 800 steps modular sum reads 0.291; at 2400 it reads **0.776**. I re-measured every number in Nova.

Hypothesis 3 — composition. If the conflict is just about which *two* things I combined, then combining all three should fix it.

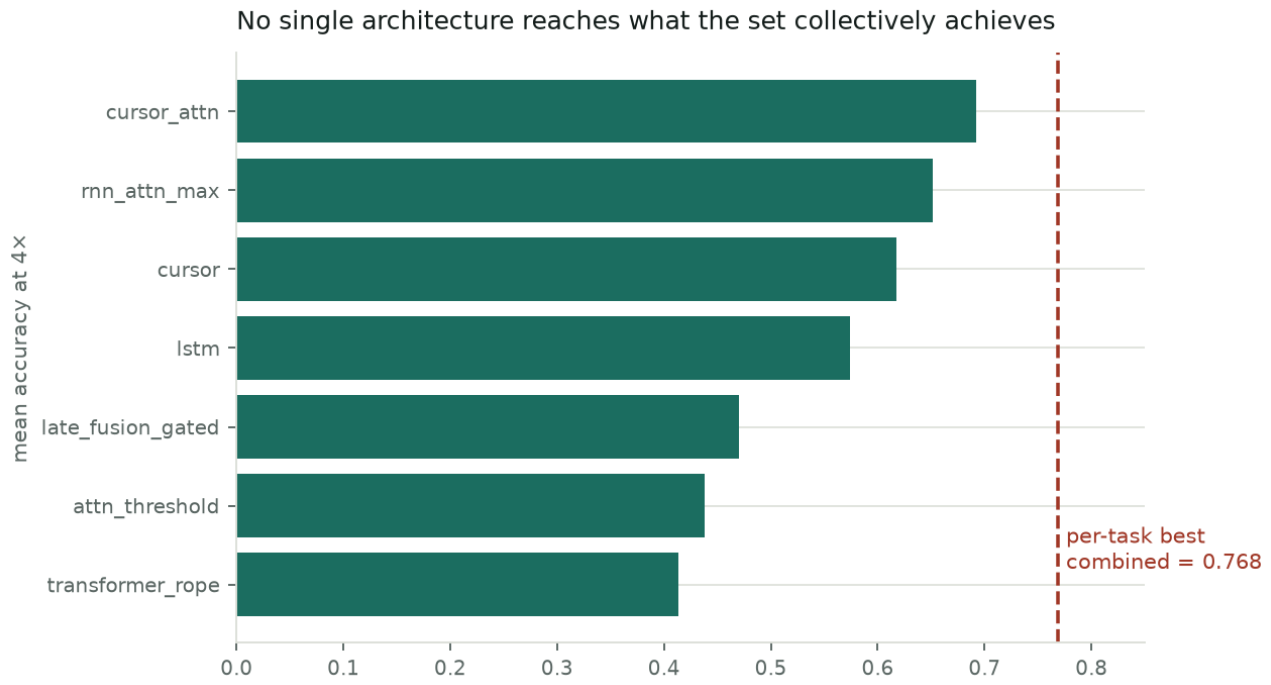
The prediction failed, but the third clause passed and that is the interesting part. Adding the third route relieved the conflict exactly as I predicted — modular sum 0.776 → **0.998** with lookup still at 0.977 — and **destroyed a different capability in the same change**: reverse fell from 0.348 to 0.146, which is chance.

> ***The conflict does not resolve. It moves.***

The frontier, and the verdict



Every architecture, every task, at four times the trained length



No single model reaches what the set collectively achieves

My best model gets 0.692. The best-per-task numbers, combined, would be 0.768. And two capabilities — copy and reverse — are unsolved by *everything*, sitting near a chance level of 0.126.

And the winner was a 2014 paper

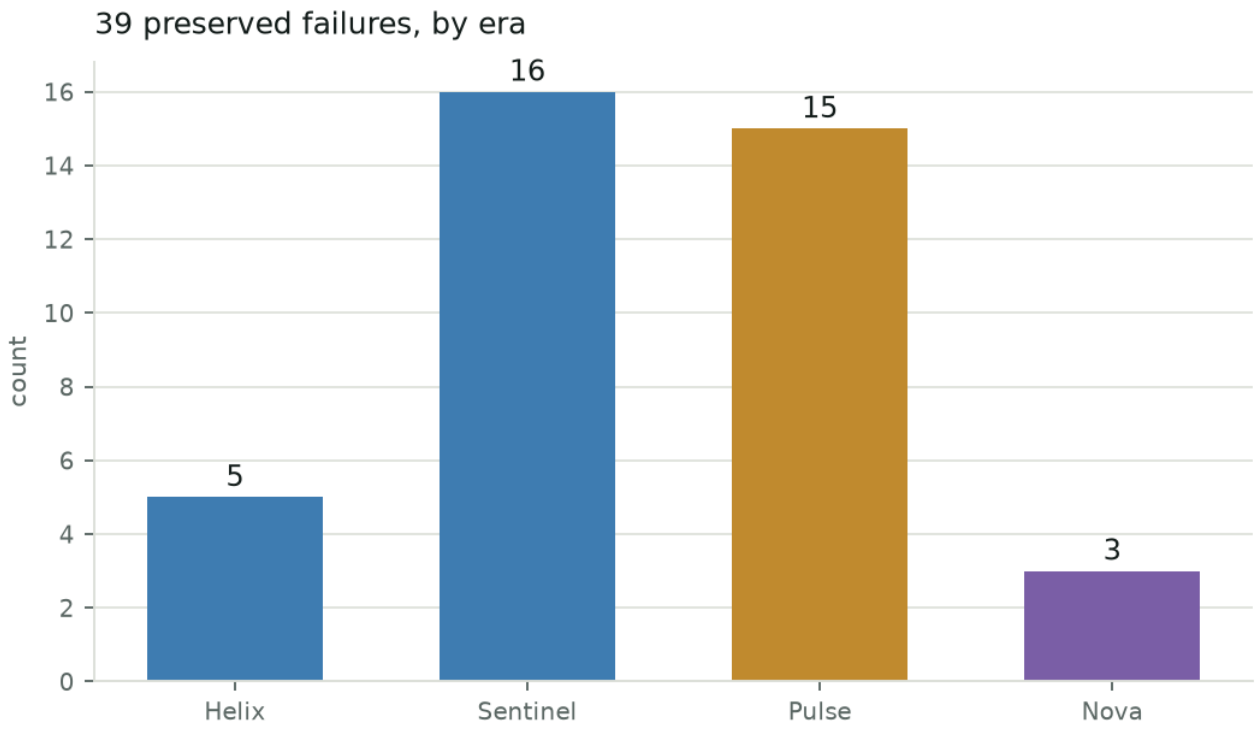
The single best mechanism I found is a "cursor": a read pointer that moves by ±1 over memory, controlled by the network.

That is **Neural Turing Machine location-based addressing**, published by Graves, Wayne and Danihelka in 2014, section 3.3.2. And copying with generalisation to longer sequences is that paper's *first experiment*. My version is a weaker copy of it, with read-only memory.

I found this before claiming anything, because I run the prior-art check *before* the comparison rather than after. That ordering is the only reason I did not embarrass myself.

---

6. So what do I actually have?



Thirty-nine failures, by era

Nineteen frozen predictions. One passed. Thirty-nine preserved failures, each with the mechanism that killed it.

What I am confident about

1 **The lockstep ceiling.** `E[max]` over a batch, not `E[halt]`. It is arithmetic, it applies to

every per-example adaptive-compute method, and it is measurable in advance.

1 **The silent failure mode.** An adaptive model whose step counter is also its answer can be

catastrophically wrong while looking perfectly healthy. Never report a compute saving without a matched accuracy number and a seed reliability rate.

1 `d_free = max(0, P − g − n(C−1))`**.** Elementary, known, and it turned seven mysterious failures

into one predicted consequence.

1 **Supervised halting works where the task supplies a stopping condition,** and loses to 2016

methods where it does not.

1 **Capability competition appears to be conserved** — this one is tentative, three seeds, and I

have an imperfect control for capacity.

What I do not have

No new mechanism. No new architecture. No capability leap. Nothing that would justify asking anyone for GPU time — you do not ask for compute to scale someone else's 2014 paper.

The scores I would give myself

Dimension	Score	Why
Reproducibility	9/10	`make reproduce-nova` re-derives it; hashes verified from disk
Publication readiness	7/10	The negative results are clean; there is no positive claim
Efficiency work	5/10	Real measured gains, narrow scope
Engineering value	5/10	Runtime characterisation is useful; the architecture is not
Capability	3/10	Nothing new works
Theoretical importance	3/10	One diagnostic, no principle
Novelty	**2/10**	My best idea is from 2014

---

7. What I would tell someone starting this

The mistakes are the transferable part, so here they are.

Build the measuring instrument before the theory. Two of my own tasks were broken and I only found out because I checked what a stupid predictor could score.

Every candidate needs a control that differs by exactly one thing. My biggest near-miss was an operator that was algebraically identical to the one it was supposed to beat.

Write down what would prove you wrong, hash it, and open it once. Nineteen times. One pass. The discipline is the whole point — without it I would have "discovered" at least four things.

A result that only predicts its own success cannot be caught being wrong. The single prediction that passed did so because one of its clauses said *where the effect stops*, and I checked that too.

Check your baselines are actually trying. Helix died because I finally tuned the GRU.

Check your training budget before your hypothesis. I nearly published an interference effect that was three times larger than reality because I had trained for too few steps.

When something looks beautiful, attack it harder. The 14-order-of-magnitude cliff, the clean power law, the 6-of-16 bifurcation count, the 1.77× speedup — all beautiful, all dead.

---

8. Honest closing

I set out to find a new principle of neural computation. I did not find one.

What I have instead is a set of measured boundaries, a pile of carefully documented failures, and a process that caught me being wrong nineteen times out of nineteen tries at being right. The one prediction that survived is narrow and its ceiling is known.

I am aware that "I failed carefully" is not a thrilling headline. But I would rather be the person with 39 documented failures and one honest result than the person with an exciting claim that falls apart when someone checks. The whole thing is reproducible — hashes, seeds, environment, one command — so if I am wrong about any of it, that should be findable.

Two things stay open, and I think they are the good ones:

- **Nothing extrapolates on copy or reverse.** Best is 0.470 against chance 0.126, and there is a

2014 method that reportedly does better. Reimplementing it faithfully is the obvious next thing.

- **Is capability competition fundamental, or is it just that I ran out of width?** Three routes at a

fixed parameter budget each get a third of the room. My control for that is approximate, and it is the criticism I would make first if I were reading this.